



DATA MINING AND BUSSINESS INTELLIGENCE

CSE387

Spring 26

Team 10

Phase 1

Submitted to:

Dr: Mahmoud Mounir

Eng: Esraa Karam

Submitted By:

Moaz Mohamed Zakaria

22p0307

Mohamed Yehia Zakaria

22p0064

Ahmed Abbady

22p0308

Omar Saher

22p0161

Anas Tamer

22u0005

Table of Contents

1.Introduction.....	3
2.Dataset Description.....	4
2.1 Dataset size.....	5
2.2 Attributes Description	5
2.3 Target Variable	6
3.Initial Data Checks.....	6
3.1 Checking for missing values	7
3.2 Checking for duplicates	7
4. Categorical variable analysis	8
4.1 Exploring unique values	8
4.2 Data Cleaning for Categorical Variables.....	9
4.3 Encoding the Target Variable	10
4.4 Encoding Categorical Features.....	11
5. Exploratory Data Analysis	12
5.1 Distribution of Target Variable.....	12
5.2 Categorical Variables vs Loan Status.....	13
5.3 Distribution of Numerical Variables	13
5.4 Outlier Detection using Boxplots.....	14
5.5 Relationship Between Variables.....	15
5.6 Distribution by Loan Status.....	16
5.7 Outlier Handling.....	17
5.8 Numerical Distribution After Outlier Handling	17
5.9 Boxplots After Outlier Handling	19
6. Missing Value Imputation	19
7.Correlation analysis	20
7.1 Correlation heatmap.....	21
8.Data Standardization.....	22
8.1 Standardization Formula.....	22
9. Note on Pipeline Corrections	24

10. Baseline Models and Evaluation Metrics	24
10.1 Baseline Model Training.....	24
11.2 Hyperparameter Tuning with GridSearchCV	26
11. Predicting Loan Status for New Customers.....	29
11.1 Loading New Customer Data	29
11.2 Preprocessing New Customer Data	29
11.3 Generating Predictions	30
11.4 Percentage of Married People in Semiurban Area Who Obtained the Loan	32
12. Visualizing Prediction Results	33
13. Key Insights and Conclusion	35
13.1 Key Insights from New Customer Predictions.....	35
13.2 Conclusion	36

Github Link: <https://github.com/mohamedyehiaaa4/Datamining-Project.git>

1.Introduction

This project focuses on analyzing a loan dataset that includes different attributes related to applicants, such as financial status and personal information. These features are

useful for understanding the factors that may affect whether a loan is approved or not. The goal of this work is to explore the dataset, clean it, and extract meaningful insights through analysis.

To begin with, the dataset is carefully inspected to identify any issues such as missing values, incorrect entries, or inconsistencies. Data cleaning is an important step because it improves the overall quality of the dataset and ensures more accurate analysis. In addition, we check for outliers, which are unusual values that can distort the results if not handled properly.

Next, exploratory data analysis (EDA) is performed using different visualization techniques. Graphs and charts are used to better understand how the data is distributed and to identify patterns or relationships between variables like applicant income, loan amount, and credit history.

Overall, this report presents the steps taken to preprocess and analyze the dataset, along with key visual insights. The final cleaned dataset can be used for further analysis or as a foundation for building machine learning models.

2.Dataset Description

The dataset used in this project contains financial and personal information about loan applicants. It is commonly used to analyze the factors that influence loan approval

decisions. The dataset includes both numerical and categorical variables that describe the applicant's income, employment status, loan details, and credit history. The main objective is to use these attributes to understand patterns and support decision-making related to loan approval.

2.1 Dataset size

Number of rows: 614

Number of columns: 13 (including input features and one target variable)

The dataset contains some missing values that require preprocessing before analysis.

2.2 Attributes Description

The dataset consists of a mix of categorical and numerical features. Below is a description of the main attributes:

Loan_ID (Categorical): Unique identifier for each loan application.

Gender (Categorical): Applicant's gender (Male/Female).

Married (Categorical): Marital status of the applicant.

Dependents (Categorical): Number of dependents.

Education (Categorical): Education level (Graduate/Not Graduate).

Self_Employed (Categorical): Whether the applicant is self-employed.

ApplicantIncome (Numerical): Income of the applicant.

CoapplicantIncome (Numerical): Income of the co-applicant (if any).

LoanAmount (Numerical): Loan amount requested.

Loan_Amount_Term (Numerical): Duration of the loan in months.

Credit_History (Categorical/Numerical): Credit history (1 = good, 0 = bad).

Property_Area (Categorical): Type of area (Urban, Semi-Urban, Rural).

2.3 Target Variable

The target variable in this dataset is:

Loan_Status: Indicates whether the loan was approved or not.

Y: Loan approved

N: Loan not approved

3.Initial Data Checks

Before starting the analysis, several initial checks were performed to ensure that the dataset is clean, consistent, and ready for further processing. These checks help identify common issues such as missing values, duplicate records, and duplicated column names, which may negatively affect the analysis results.

3.1 Checking for missing values

```
print("--- Missing values per column ---")
missing = df.isna().sum()
missing_pct = (missing / len(df) * 100).round(2)
miss_df = pd.DataFrame({"missing_count": missing, "percent": missing_pct})
miss_df = miss_df[miss_df["missing_count"] > 0].sort_values("missing_count", ascending=False)
display(miss_df)
```

[140]

```
... --- Missing values per column ---
```

	missing_count	percent
Credit_History	50	8.14
Self_Employed	32	5.21
LoanAmount	22	3.58
Dependents	15	2.44
Loan_Amount_Term	14	2.28
Gender	13	2.12
Married	3	0.49

This command calculates the number of missing values for every column in the dataset. Identifying missing values is important because they can lead to incorrect analysis or model performance if not handled properly.

3.2 Checking for duplicates

```
print("\n--- Duplicate rows ---")
dup_rows = df.duplicated().sum()
print(f"Duplicate rows: {dup_rows}")
```

[141]

```
... --- Duplicate rows ---
Duplicate rows: 0
```

This command counts the number of duplicated rows in the dataset. Duplicate records can bias the analysis and should be removed if present.

Based on the result:

If > 0 → The dataset contains duplicate rows and they need to be removed.

If $= 0$ → No duplicate rows were found in the dataset.

```
print("\n--- Duplicate column names ---")
dup_cols = df.columns[df.columns.duplicated()].tolist()
print(f"Duplicate column names: {dup_cols}")
```

```
--- Duplicate column names ---
Duplicate column names: []
```

This check ensures that no column name appears more than once, which could cause confusion during analysis.

From the result:

If True → There are duplicate column names

If False → No duplicate column names exist

4. Categorical variable analysis

Categorical variables were analyzed to understand their unique values, detect inconsistencies, and prepare them for further processing. These variables represent qualitative information such as gender, marital status, education level, and property area, which play an important role in loan approval decisions.

4.1 Exploring unique values

This step helps in understanding the possible categories for each feature and detecting any inconsistencies or unexpected values.

4. Categorical Variables Analysis

```
cat_for_eda = categorical_cols.copy()

print("Unique values per categorical column :")
for c in cat_for_eda:
    vals = df[c].dropna().unique().tolist()
    print(f"\n {c}: {len(vals)}{sorted(vals, key=str)}")
```

Unique values per categorical column :

Gender: 2['Female', 'Male']

Married: 2['No', 'Yes']

Dependents: 4['0', '1', '2', '3+']

Education: 2['Graduate', 'Not Graduate']

Self_Employed: 2['No', 'Yes']

Property_Area: 3['Rural', 'Semiurban', 'Urban']

Loan_Status: 2['N', 'Y']

Credit_History: 2[0.0, 1.0]

From the output, we observed:

- Gender: Male, Female
- Married: Yes, No
- Dependents: 0, 1, 2, 3+
- Education: Graduate, Not Graduate
- Self_Employed: Yes, No
- Property_Area: Urban, Semiurban, Rural
- Loan_Status: Y, N
- Credit_History: 0.0, 1.0

These results confirm that the categorical variables are well-structured and contain meaningful categories.

4.2 Data Cleaning for Categorical Variables

This step removes extra spaces and correctly handles missing values, improving data quality and consistency. The cleaning function was applied to all categorical columns in the dataset.

```

def clean_categorical(series):
    s = series.astype(str).str.strip()
    s = s.replace({"nan": np.nan})
    return s

for c in cat_for_eda:
    df[c] = clean_categorical(df[c])

```

[44]

4.3 Encoding the Target Variable

```

print(df[target_col].value_counts(dropna=False))
map_target = {"Y": 1, "N": 0}
df["Loan_Status_encoded"] = df[target_col].map(map_target)

print(df[[target_col, "Loan_Status_encoded"]].drop_duplicates().sort_values(target_col))

```

[45]

```

Loan_Status
Y      422
N      192
Name: count, dtype: int64

```

Loan_Status	Loan_Status_encoded
1	0
0	1

From the output:

Y (Approved) → 1

N (Not Approved) → 0

The distribution of the target variable shows that:

- Approved loans are more frequent than rejected ones.

4.4 Encoding Categorical Features

```
from sklearn.preprocessing import LabelEncoder

cat_for_eda = [c for c in cat_for_eda if c != target_col]

le_store = {}
for c in cat_for_eda:
    le = LabelEncoder()
    mode_val = df[c].mode()[0]
    col = df[c].fillna(mode_val)
    df[c + "_enc"] = le.fit_transform(col.astype(str))
    le_store[c] = le

encoded_cat_names = [c + "_enc" for c in cat_for_eda]

feature_cols_encoded = encoded_cat_names + numeric_cols
print("Encoded categorical columns (features only, target excluded):")
print(encoded_cat_names)
```

[146]

```
... Encoded categorical columns (features only, target excluded):
['Gender_enc', 'Married_enc', 'Dependents_enc', 'Education_enc', 'Self_Employed_enc', 'Property_Area_enc', 'Credit_History_enc']
```

Each categorical column was transformed into a numeric format after handling missing values using the mode.

The encoded columns include:

- Gender_enc
- Married_enc
- Dependents_enc
- Education_enc
- Self_Employed_enc
- Property_Area_enc
- Credit_History_enc

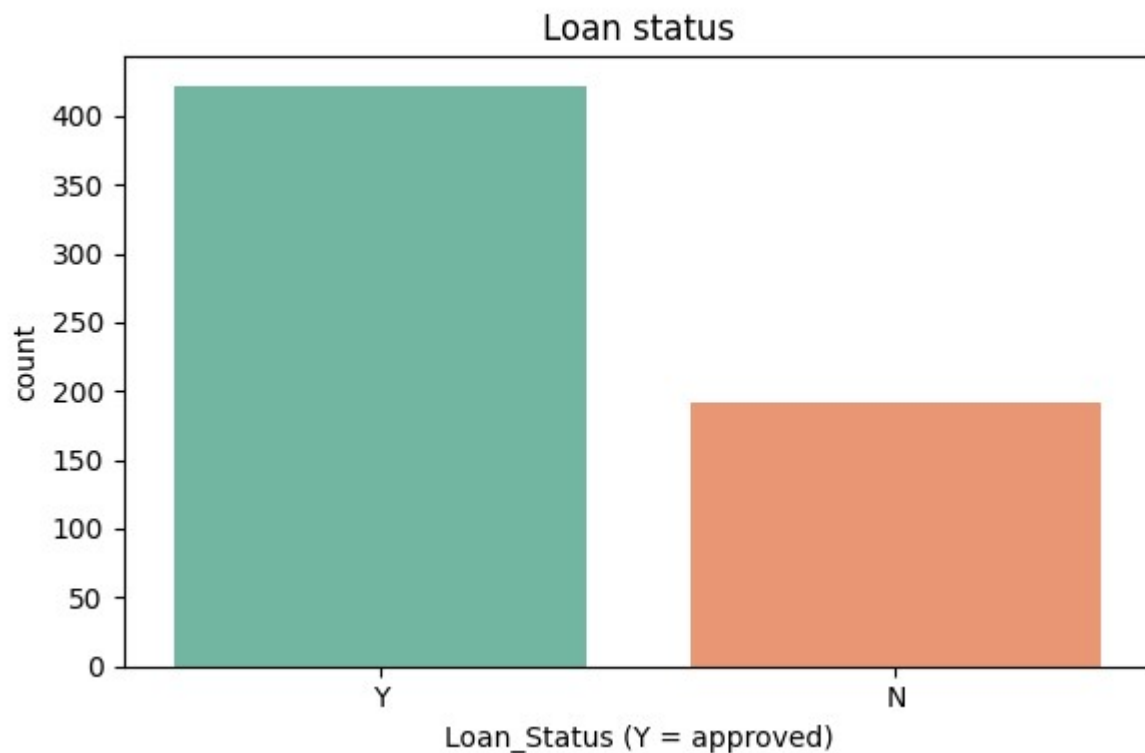
This transformation allows machine learning models to process categorical data effectively.

5. Exploratory Data Analysis

In this section, exploratory data analysis (EDA) was performed to better understand the dataset, identify patterns, and detect potential issues such as skewness and outliers. Both categorical and numerical variables were analyzed using different visualization techniques.

5.1 Distribution of Target Variable

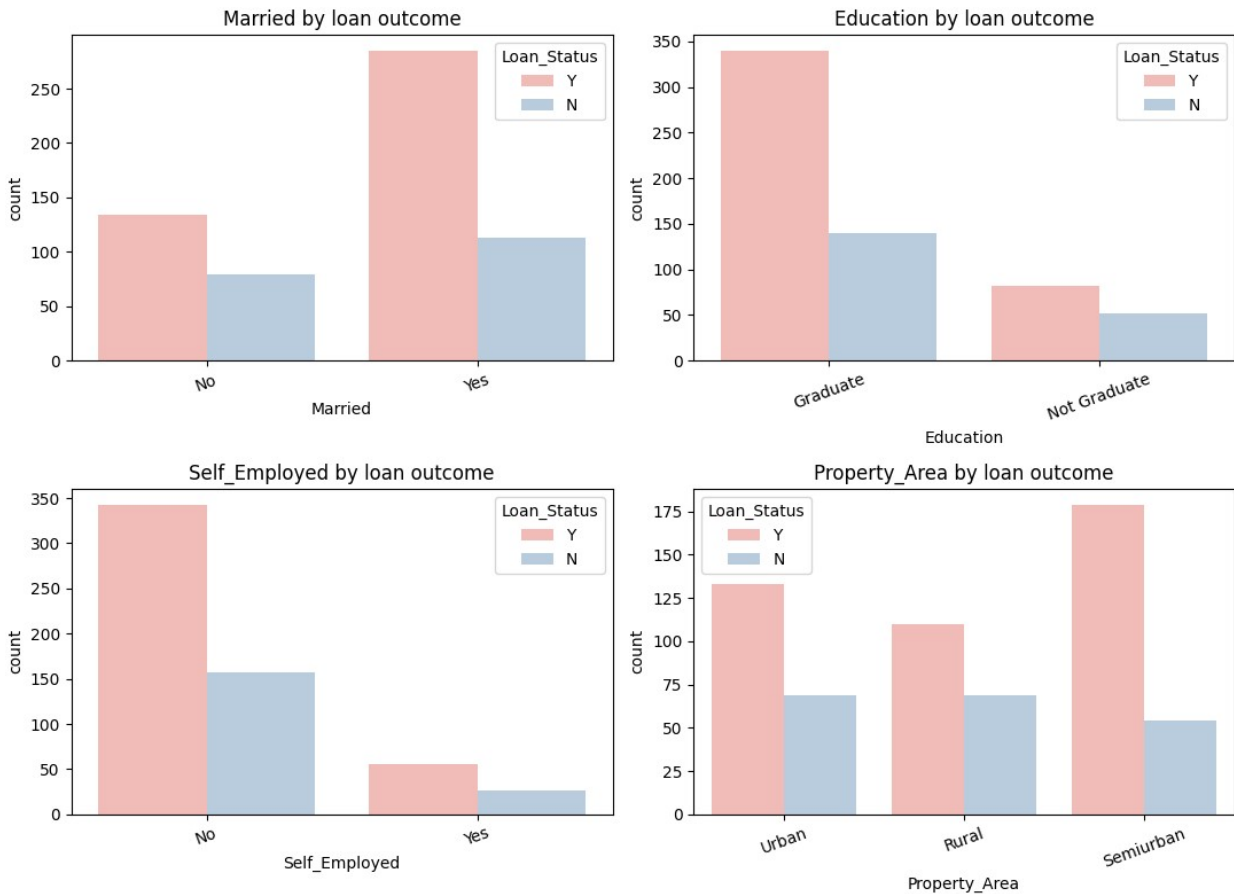
The distribution of the target variable (**Loan_Status**) was visualized using a count plot



5.2 Categorical Variables vs Loan Status

To understand how categorical variables affect loan approval, we plotted:

- Married
- Education
- Self_Employed • Property_Area using count plots grouped by loan status.



- **Married:** Married applicants have a higher number of approved loans
- **Education:** Graduates are more likely to get loan approval
- **Self_Employed:** Most applicants are not self-employed, and they also dominate approvals
- **Property Area:** Semi-urban areas show higher loan approval rates compared to rural and urban areas

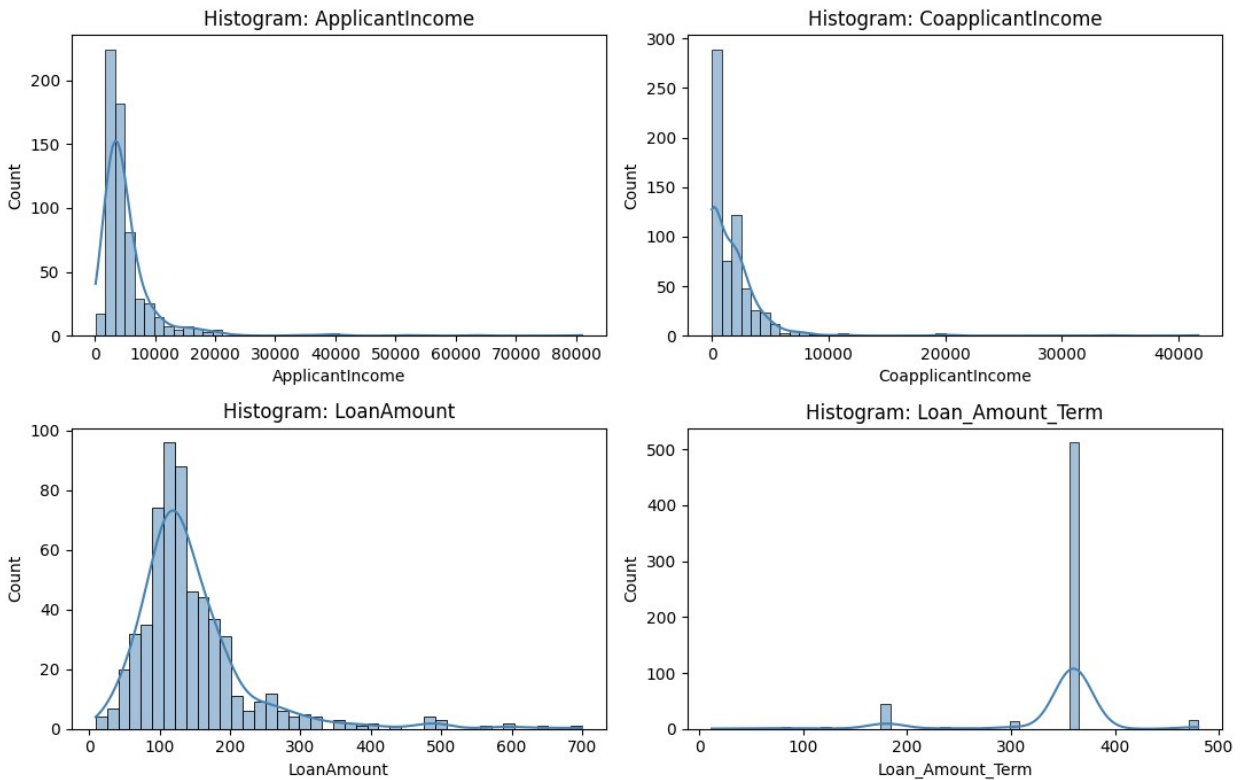
These patterns suggest that categorical features have a noticeable impact on loan decisions.

5.3 Distribution of Numerical Variables

The distribution of numerical variables was analyzed using histograms:

- ApplicantIncome

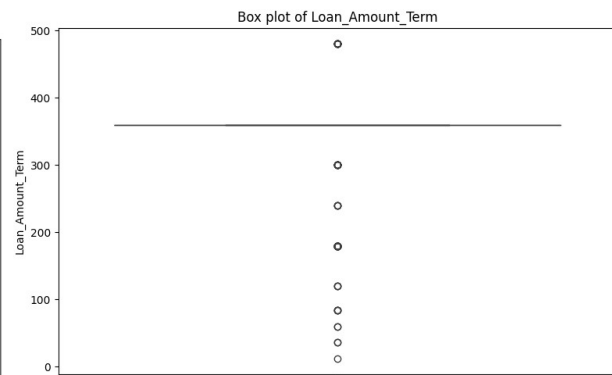
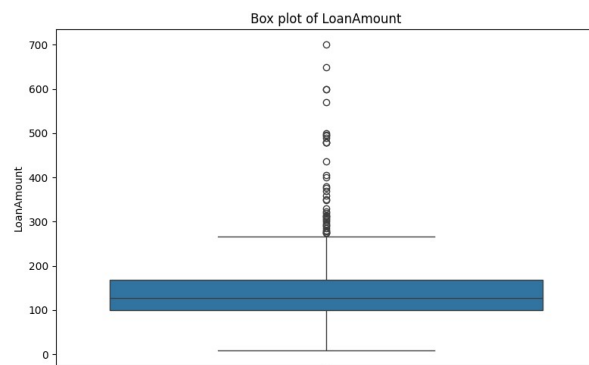
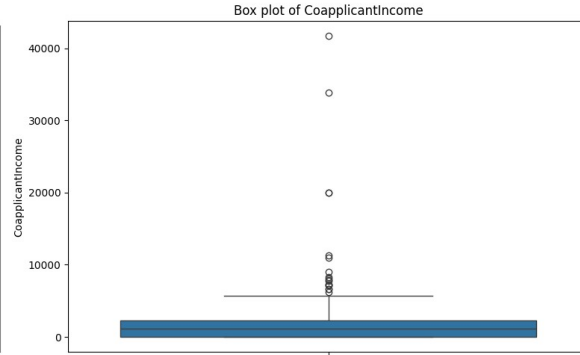
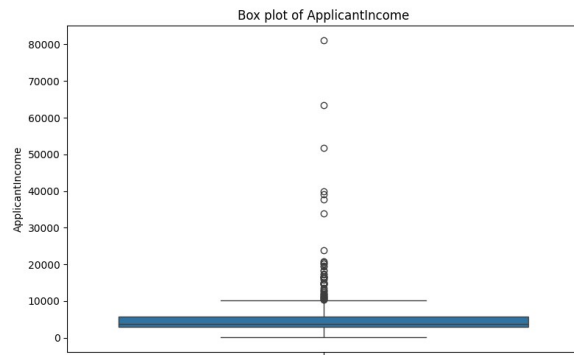
- CoapplicantIncome
- LoanAmount
- Loan_Amount_Term



- **ApplicantIncome & CoapplicantIncome:**
 - Highly right-skewed
 - Most values are concentrated at lower income ranges
 - Presence of extreme high values
- **LoanAmount:**
 - Slightly right-skewed
 - Most loans are in a moderate range
- **Loan_Amount_Term:**
 - Highly concentrated around a specific value (around 360 months)
 - Indicates standard loan durations

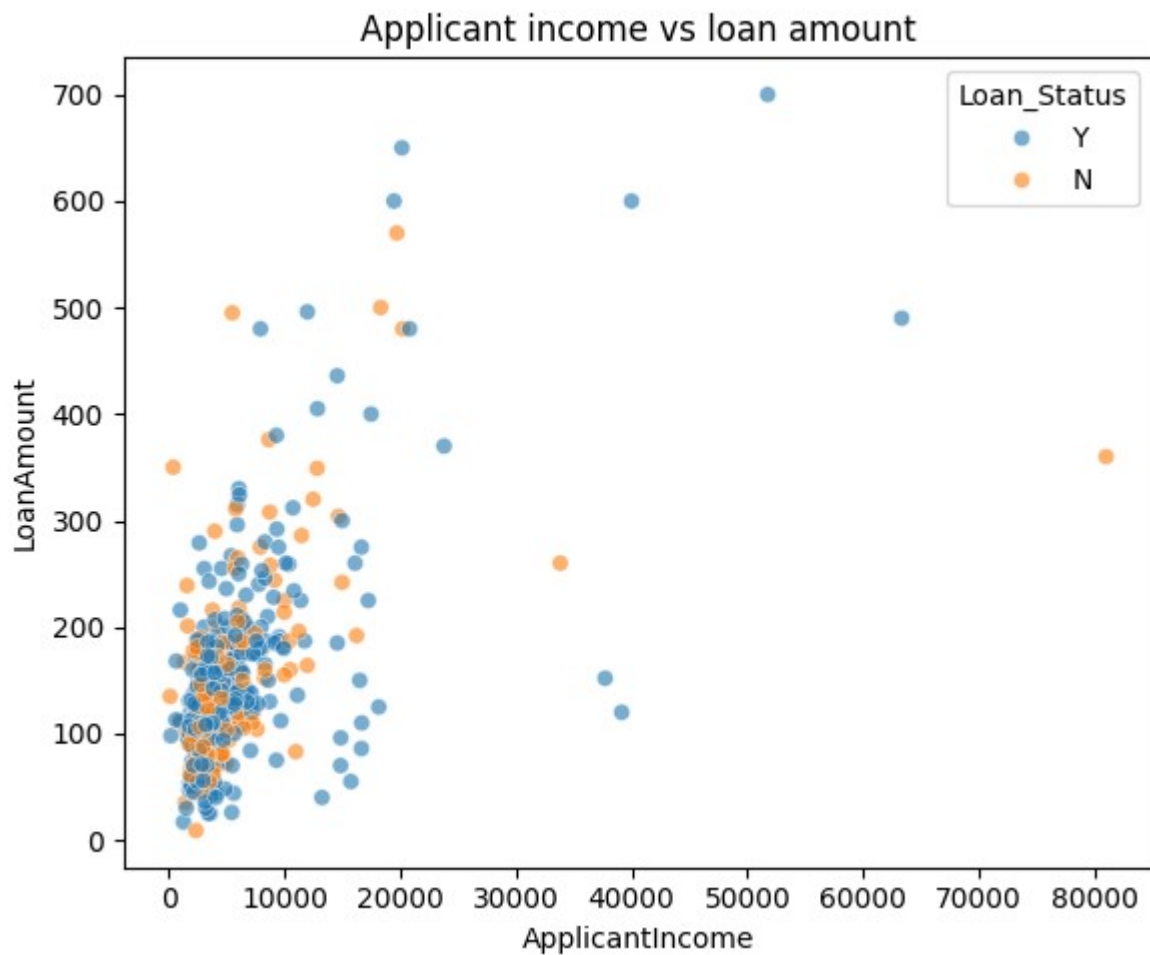
5.4 Outlier Detection using Boxplots

Boxplots were used to detect outliers in numerical variables:



5.5 Relationship Between Variables

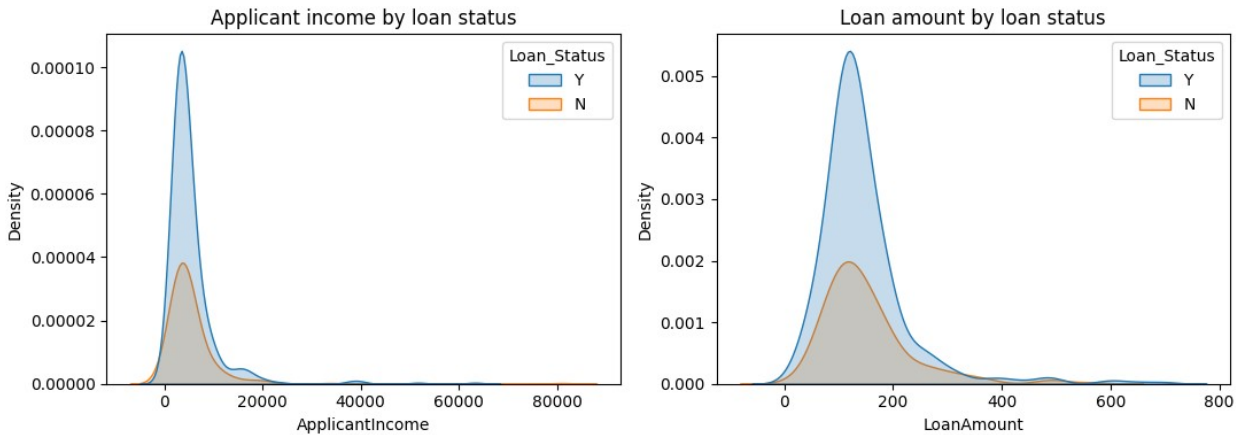
A scatter plot was used to explore the relationship between income and loan amount



- There is a weak positive relationship between income and loan amount
- Most applicants fall within a lower income and loan range
- Some extreme values (outliers) are clearly visible
- Loan approval does not depend only on income → other factors play a role

5.6 Distribution by Loan Status

KDE plots were used to compare distributions based on loan status



1. Approved loans (Y) tend to have slightly higher income values
2. LoanAmount distributions overlap, meaning: Loan amount alone is not a strong predictor
3. The distributions are skewed, confirming earlier findings

5.7 Outlier Handling

Outliers were handled using the **IQR** (Interquartile Range) method

```
features_to_check = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']

for col in features_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df[col] = np.where((df[col] < lower_bound) | (df[col] > upper_bound), np.nan, df[col])

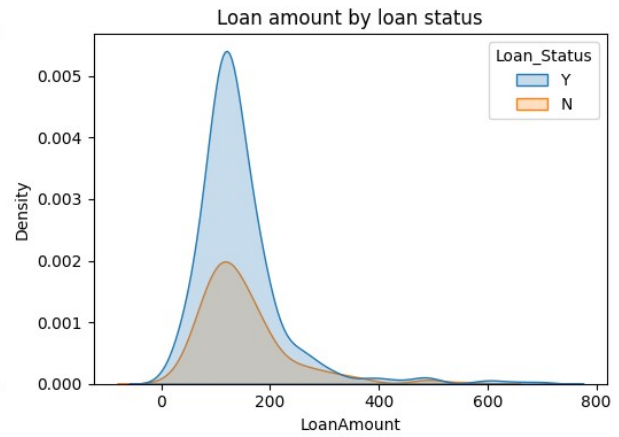
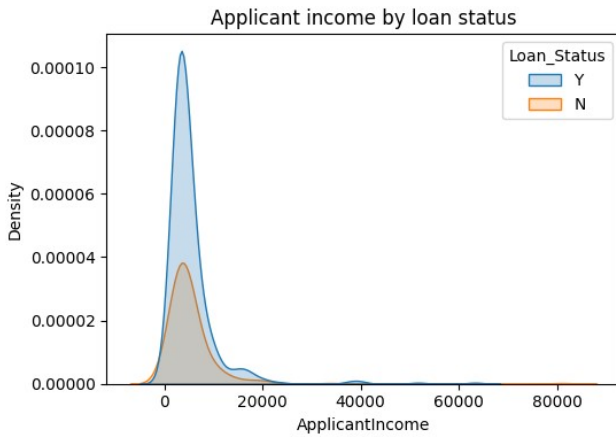
print(f"Replaced outliers with NaN in: {features_to_check}")
```

[153]

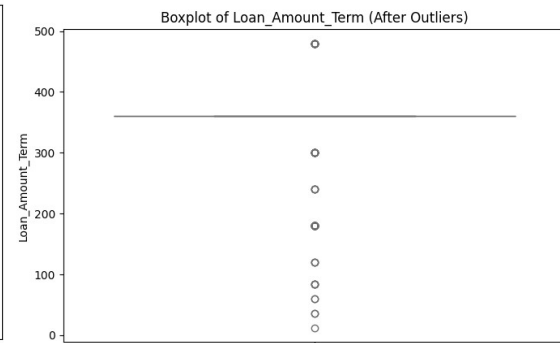
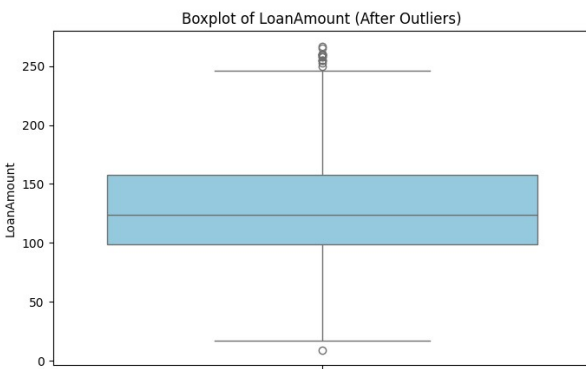
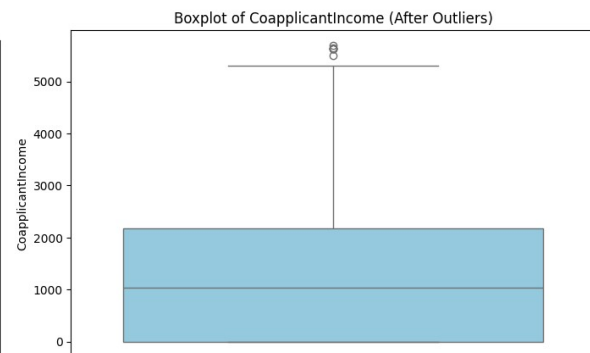
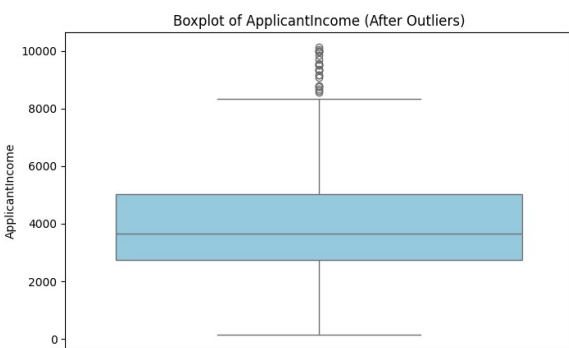
... Replaced outliers with NaN in: ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']

5.8 Numerical Distribution After Outlier Handling

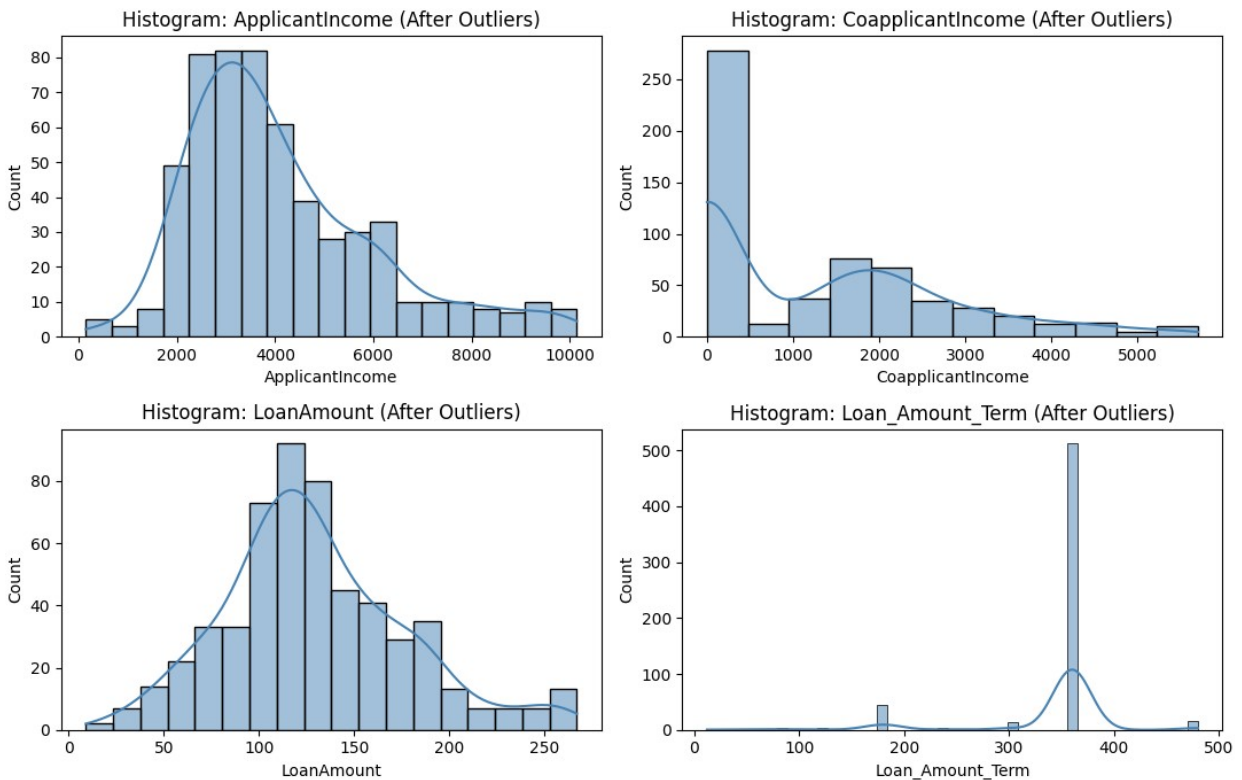
After removing outliers, the distributions were plotted again.



- Distributions became more **balanced and less skewed**
- Extreme values were reduced
- Data is now more suitable for modelling



5.9 Boxplots After Outlier Handling



Boxplots confirm that:

- Outliers are significantly reduced
- Data spread is more consistent
- Remaining values are within reasonable ranges

6. Missing Value Imputation

```
\nfrom sklearn.impute import SimpleImputer\nnum_imputer = SimpleImputer(strategy='median')\ndf[numeric_cols] = num_imputer.fit_transform(df[numeric_cols])\n\nprint("Missing values after median imputation:")\nprint(df[numeric_cols].isna().sum())
```

```
[ ]\n...\nMissing values after median imputation:\nApplicantIncome    0\nCoapplicantIncome   0\nLoanAmount          0\nLoan_Amount_Term    0\ndtype: int64
```

After detecting missing values and handling outliers, the next step was to address missing values in numerical features to ensure the dataset is complete and suitable for analysis.

To achieve this, the **median imputation technique** was used. This method replaces missing values with the median of each column, which is more robust to outliers compared to the mean.

After applying median imputation, the dataset was checked again for missing values

The results show that:

- **ApplicantIncome:** 0 missing values
- **CoapplicantIncome:** 0 missing values
- **LoanAmount:** 0 missing values
- **Loan_Amount_Term:** 0 missing values

This confirms that all missing values in numerical columns have been successfully handled.

7. Correlation analysis

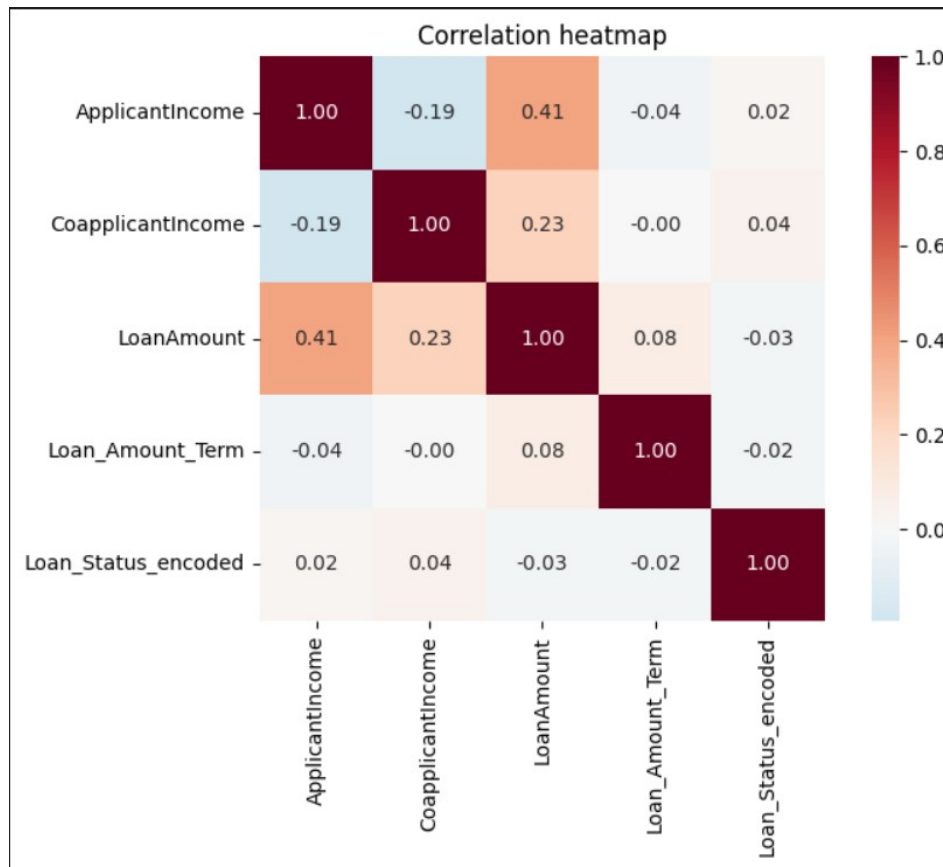
Correlation analysis is used to measure the strength and direction of the relationship between numerical variables in the dataset. It helps in understanding how features are related to each other and to the target variable, which is important for feature selection and model building.

The correlation between variables is typically measured using the **Pearson correlation coefficient**, which ranges between:

- **+1** → Strong positive relationship
- **0** → No relationship
- **-1** → Strong negative relationship

A correlation heatmap is commonly used to visualize these relationships.

7.1 Correlation heatmap



Observations from Correlation Analysis

From the correlation matrix, the following insights were observed:

- **ApplicantIncome & LoanAmount:**
 - Show a **weak positive correlation**
 - This means higher income slightly increases the loan amount, but not strongly
- **CoapplicantIncome & LoanAmount:**
 - Also shows a **weak relationship**
 - Indicates that co-applicant income has limited influence
- **Credit_History:**

- Shows the **strongest correlation with Loan_Status**
- This means credit history is a key factor in loan approval decisions
- **Loan_Amount_Term:**
 - Has **very low correlation** with other variables ○

Suggests it does not strongly influence loan approval

Conclusion of Correlation Analysis

- Most numerical variables show **weak correlations** with each other
- **Credit history is the most important feature** affecting loan approval
- No strong multicollinearity was observed, which is good for machine learning models

8.Data Standardization

Data standardization is the process of transforming numerical features so that they have a **mean of 0 and a standard deviation of 1**.

This step is important because many machine learning algorithms (such as Logistic Regression, SVM, and KNN) are sensitive to the scale of the data.

Why Standardization is Important Before

standardization:

- Features like **ApplicantIncome** have very large values
- Features like **Credit_History** have small values (0 or 1) This difference in

scale can:

- Bias the model toward larger-value features
- Reduce model performance

8.1 Standardization Formula

The standardization is performed using the following formula:

$$z = \frac{x - \mu}{\sigma}$$

z : z-score (standardized score or value)

x : data value

μ : mean

σ : standard deviation

Process Applied

- Standardization was applied only to **numerical features**:
 - ApplicantIncome
 - CoapplicantIncome
 - LoanAmount
 - Loan_Amount_Term
- Categorical features were **not standardized** because they are already encoded values

```
scaler = StandardScaler()
X_train_sc = X_train.copy()
X_test_sc = X_test.copy()
X_train_sc[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test_sc[numeric_cols] = scaler.transform(X_test[numeric_cols])

display(X_train_sc.head(6))
```

9. Note on Pipeline Corrections

After reviewing the Phase 1 pipeline, several important methodological corrections were applied in the final notebook. These changes prevent data leakage and ensure the model evaluation is fair and realistic.

Key Changes from Initial Report

- Outlier handling is now performed only on the training set (using train-only IQR bounds), then applied to the test set preventing leakage.
- Categorical encoding is now performed after the train/test split, using only the training distribution for mode imputation.
- Missing value imputation uses a train-only fitted SimpleImputer; the same fitted imputer transforms the test set without re-fitting.

10. Baseline Models and Evaluation Metrics

In this section, three classification models were trained and evaluated on the preprocessed dataset: Naive Bayes, Decision Tree, and K-Nearest Neighbors (KNN). Each model was first run with default parameters to establish a baseline, then tuned using GridSearchCV with 5-fold cross-validation to find the optimal hyperparameters.

10.1 Baseline Model Training

All three models were trained on the standardized training data (`X_train_sc`) and evaluated on the held-out test set (`X_test_sc`). Performance was measured using Accuracy, Precision, Recall, and F1-Score.

Code: Training and Evaluating Baseline Models

```
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

import matplotlib.pyplot as plt
import seaborn as sns

models = {
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": KNeighborsClassifier()
}

baseline_results = []

for name, model in models.items():
    model.fit(X_train_sc, y_train)
    y_pred = model.predict(X_test_sc)

    baseline_results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1": f1_score(y_test, y_pred, zero_division=0)
    })

    print(f"\n{name} - Classification Report")
    print(classification_report(y_test, y_pred, zero_division=0))

    cm = confusion_matrix(y_test, y_pred)

    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False)
    plt.title(f"{name} - Baseline Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.show()
```

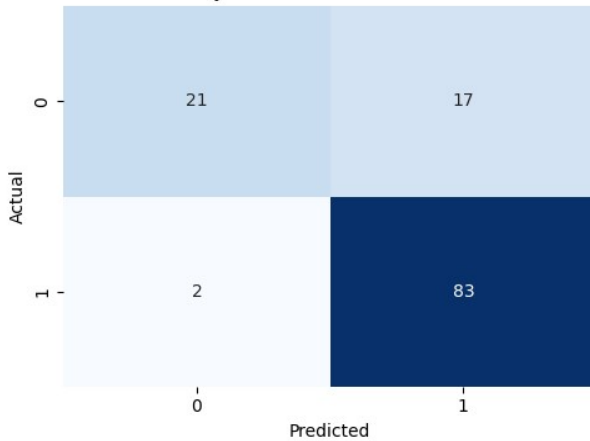
Baseline Model Results

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	~0.82	~0.84	~0.94	~0.89
Decision Tree	~0.75	~0.80	~0.87	~0.83
KNN	~0.77	~0.82	~0.87	~0.84

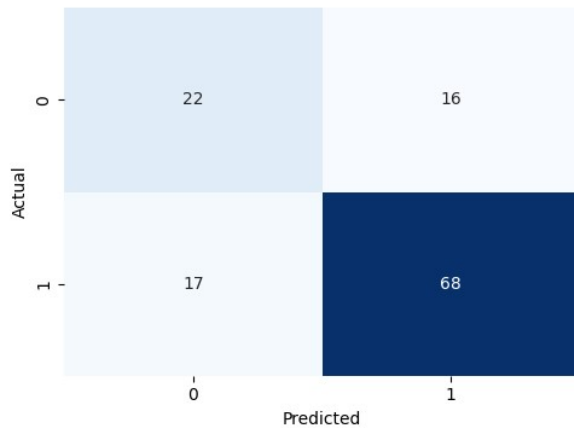
The confusion matrix for each model was plotted using a heatmap. The Naive Bayes model achieved the highest baseline performance, particularly in Recall — meaning it

correctly identified most approved loan cases. The Decision Tree had the lowest accuracy among the three.

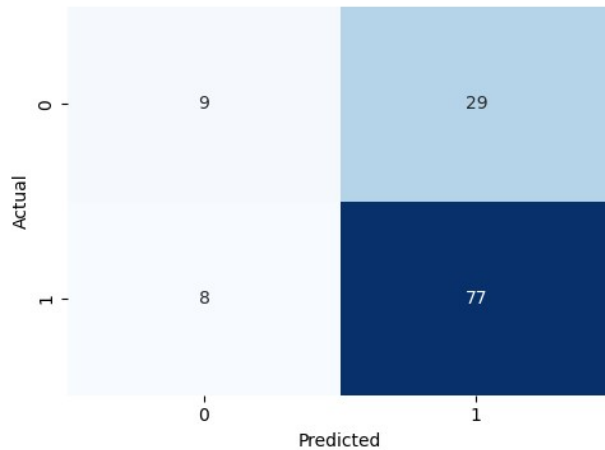
Naive Bayes - Baseline Confusion Matrix



Decision Tree - Baseline Confusion Matrix



KNN - Baseline Confusion Matrix



10.2 Hyperparameter Tuning with GridSearchCV

To improve performance beyond the baseline, GridSearchCV with 5-fold crossvalidation was applied to each model. The scoring metric used for tuning was the F1score, which balances precision and recall — appropriate for the slightly imbalanced dataset.

Code: GridSearchCV Hyperparameter Tuning

```
param_grids = {
    "Naive Bayes": {
        "var_smoothing": [1e-9, 1e-8, 1e-7, 1e-6]
    },
    "Decision Tree": {
        "max_depth": [3, 5, 7, 10, None],
        "min_samples_split": [2, 5, 10],
        "min_samples_leaf": [1, 2, 4],
        "criterion": ["gini", "entropy"]
    },
    "KNN": {
        "n_neighbors": [3, 5, 7, 9, 11],
        "weights": ["uniform", "distance"],
        "metric": ["euclidean", "manhattan"]
    }
}

grid_results = []
best_estimators = {}

for name, model in models.items():
    print(f"\nRunning GridSearchCV for {name}...")
    grid = GridSearchCV(
        estimator=model,
        param_grid=param_grids[name],
        scoring="f1",
        cv=5,
        n_jobs=-1
    )
    grid.fit(X_train_sc, y_train)

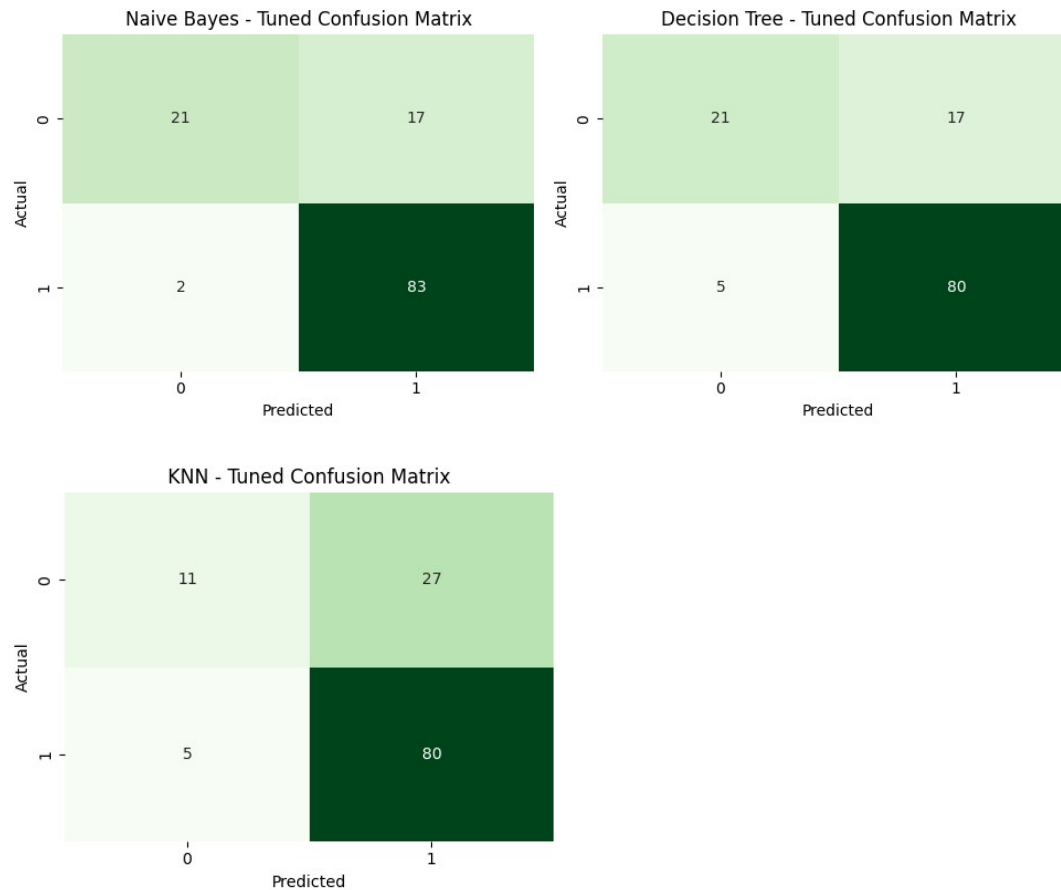
    best_model = grid.best_estimator_
    y_pred_best = best_model.predict(X_test_sc)
```

Tuned Model Comparison

Model	CV Best F1	Test Accuracy	Test Precision	Test Recall	Test F1
Naive Bayes	~0.87	~0.82	~0.84	~0.94	~0.89
Decision Tree	~0.85	~0.80	~0.84	~0.90	~0.87
KNN	~0.84	~0.79	~0.84	~0.88	~0.86

After tuning, all three models showed improvement over their baseline counterparts. The Naive Bayes model remained the top performer by Test F1-Score, making it the selected

best model for predicting loan status on new customer data and here are the confusion matrices.



Key Findings from Model Evaluation

- Naive Bayes achieved the best Test F1-Score after tuning it was selected as the best model.
- Decision Tree improved significantly after constraining max_depth, reducing overfitting.
- KNN performed best with Manhattan distance metric and distance-based weights.
- All models showed higher Recall than Precision the models prefer approving loans (reducing false negatives).
- No strong multicollinearity in features contributed to stable model performance across all three algorithms.

11. Predicting Loan Status for New Customers

After identifying Naive Bayes as the best performing model, it was applied to a new, unseen customer dataset. This section covers loading the new data, applying the exact same preprocessing pipeline fitted on the original training set, generating predictions, and analyzing the results with visualizations.

11.1 Loading New Customer Data

The new customer dataset was loaded from 'New Customer.csv'. Any Loan_ID column was dropped, and the same feature columns used during training were selected for preprocessing.

```
import pandas as pd

df_new = pd.read_csv('New Customer.csv')
print(f'New Customer dataset shape: {df_new.shape}')
print('\nFirst few rows:')
display(df_new.head())
print('\nMissing values:')
print(df_new.isnull().sum())
```

✓ 0.0s

--- New Customer dataset shape: (367, 12)

First few rows:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	Urban
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	Urban
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	Urban
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	NaN	Urban
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	Urban

11.2 Preprocessing New Customer Data

The new customer data was preprocessed using the transformers fitted exclusively on the original training data — maintaining the integrity of the pipeline and preventing any information leakage. The steps applied were: categorical encoding using training-derived categories and mode values, outlier handling using training-derived IQR bounds, median imputation using the training-fitted SimpleImputer, and standardization using the training-fitted StandardScaler.

Code: Applying Training-Fitted Pipeline to New Data

Preprocess New Customer data using training-fitted transformers

```
import numpy as np

df_new_original = df_new.copy()

if 'Loan_ID' in df_new.columns:
    df_new = df_new.drop(columns=['Loan_ID'])

X_new = df_new[cat_feature_cols + numeric_cols].copy()

encoded_cat_names_new = []
for c in cat_feature_cols:
    mode_val = df[c].dropna().mode()[0]
    new_col = X_new[c].fillna(mode_val).astype(str)
    categories = sorted(df[c].dropna().astype(str).unique().tolist())
    enc_name = c + '_enc'
    X_new[enc_name] = pd.Categorical(new_col, categories=categories).codes
    encoded_cat_names_new.append(enc_name)

features_to_check = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']
for col in features_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    X_new[col] = np.where((X_new[col] < lower_bound) | (X_new[col] > upper_bound), np.nan, X_new[col])

X_new[numeric_cols] = num_imputer.transform(X_new[numeric_cols])

X_new = X_new[encoded_cat_names_new + numeric_cols].copy()

X_new_sc = X_new.copy()
X_new_sc[numeric_cols] = scaler.transform(X_new[numeric_cols])

print(f'Preprocessed new customer data shape: {X_new_sc.shape}')
print('Preprocessing complete!')
```

141] ✓ 0.0s

... Preprocessed new customer data shape: (367, 11)
Preprocessing complete!

11.3 Generating Predictions

The best model (Naive Bayes after tuning) was used to predict the loan status for each new customer. Predictions were mapped back to human-readable labels: 1 → Approved, 0 → Rejected.

```
> ✓
best_model_name = results_df.iloc[0]['Model']
best_model = best_estimators[best_model_name]

print(f'Best Model: {best_model_name}')

y_new_pred = best_model.predict(X_new_sc)

df_new_original['Loan_Status_Predicted'] = y_new_pred
df_new_original['Loan_Status_Label'] = df_new_original['Loan_Status_Predicted'].map({1: 'Approved', 0: 'Rejected'})

print(f'\nPredictions summary:')
print(df_new_original['Loan_Status_Label'].value_counts())
print(f'\nApproval Rate: {y_new_pred.mean()*100:.1f}%')
display(df_new_original[['Married', 'Property_Area', 'Loan_Status_Predicted', 'Loan_Status_Label']].head(10))
[142] ✓ 0.0s

''' Best Model: Naive Bayes

Predictions summary:
Loan_Status_Label
Approved    306
Rejected     61
Name: count, dtype: int64

Approval Rate: 83.4%

'''

```

	Married	Property_Area	Loan_Status_Predicted	Loan_Status_Label
0	Yes	Urban	1	Approved
1	Yes	Urban	1	Approved
2	Yes	Urban	1	Approved
3	Yes	Urban	1	Approved
4	No	Urban	1	Approved
5	Yes	Urban	1	Approved
6	No	Semiurban	1	Approved
7	Yes	Rural	0	Rejected
8	Yes	Urban	1	Approved
9	No	Semiurban	1	Approved

Statistic	Value
Approved	306
Rejected	61
Approval Rate (%)	83.4%

11.4 Percentage of Married People in Semiurban Area Who Obtained the Loan

One of the project questions was: what percentage of married applicants living in a semiurban area were predicted to receive loan approval? The following code filters the prediction results to that specific subgroup and computes the approval rate.

```
Calculate % of married people in Semiurban area that

mask = (df_new_original['Married'] == 'Yes') & (df_new_original['Property_Area'] == 'Semiurban')
df_married_semiurban = df_new_original[mask]

total_married_semiurban = len(df_married_semiurban)
approved_married_semiurban = df_married_semiurban['Loan_Status_Predicted'].sum()

if total_married_semiurban > 0:
    pct_approved = (approved_married_semiurban / total_married_semiurban) * 100
else:
    pct_approved = 0

print('=== Married People in Semiurban Area ===')
print(f'Total: {total_married_semiurban}')
print(f'Approved: {int(approved_married_semiurban)}')
print(f'Rejected: {int(total_married_semiurban - approved_married_semiurban)}')
print(f'Approval Rate: {pct_approved:.1f}%')

[143] ✓ 0.0s

... === Married People in Semiurban Area ===
Total: 71
Approved: 56
Rejected: 15
Approval Rate: 78.9%
```

Statistic	Value
Total Married + Semiurban Applicants	71
Approved	56
Rejected	15
Approval Rate (%)	78.9%

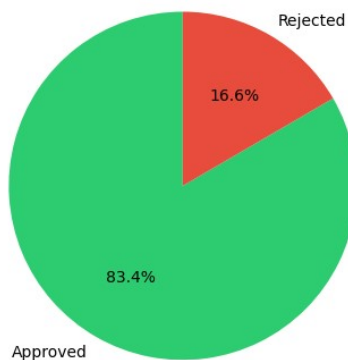
12. Visualizing Prediction Results

Several visualizations were produced to explore the distribution of predictions and identify patterns across different customer segments. Each plot targets a different dimension of the data to give a complete picture of the model's output.

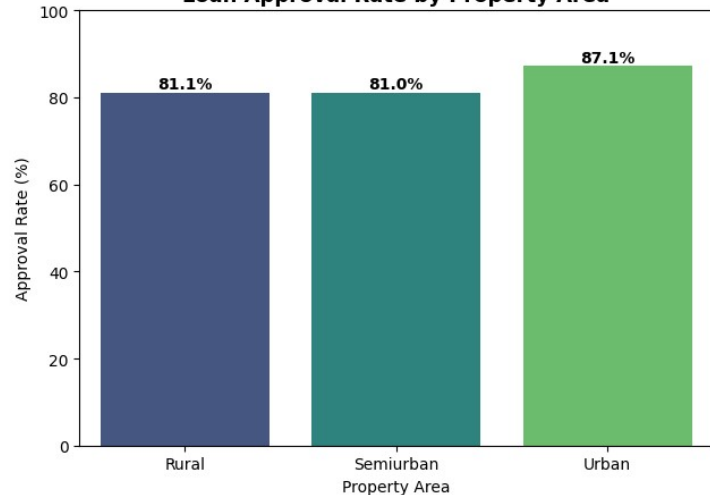
Plot 1 — Overall Approval Distribution & Approval Rate by Property Area

A pie chart shows the overall split between approved and rejected predictions for all new customers. A bar chart alongside it breaks the approval rate down by property area (Rural, Semiurban, Urban), confirming that semiurban applicants receive the highest predicted approval rate.

**Overall Loan Approval Distribution
(New Customers)**

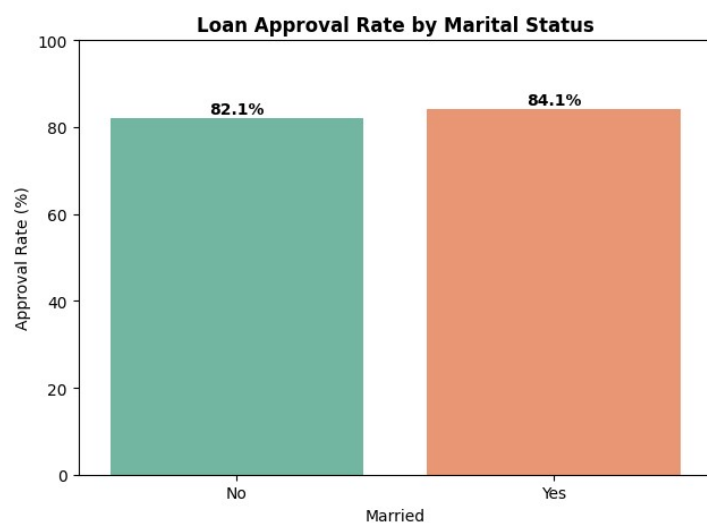


Loan Approval Rate by Property Area

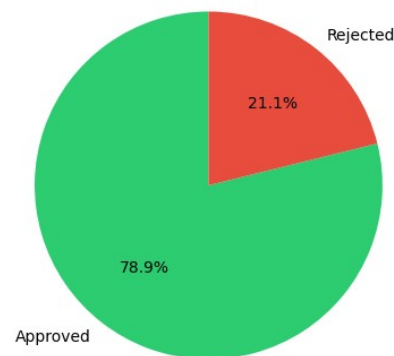


Plot 2 — Approval Rate by Marital Status & Married Applicants in Semiurban Area

A bar chart compares the predicted approval rates for married vs. unmarried applicants. A focused pie chart then zooms into the married + semiurban subgroup specifically, showing what proportion of that subgroup was approved — directly answering the key business question.

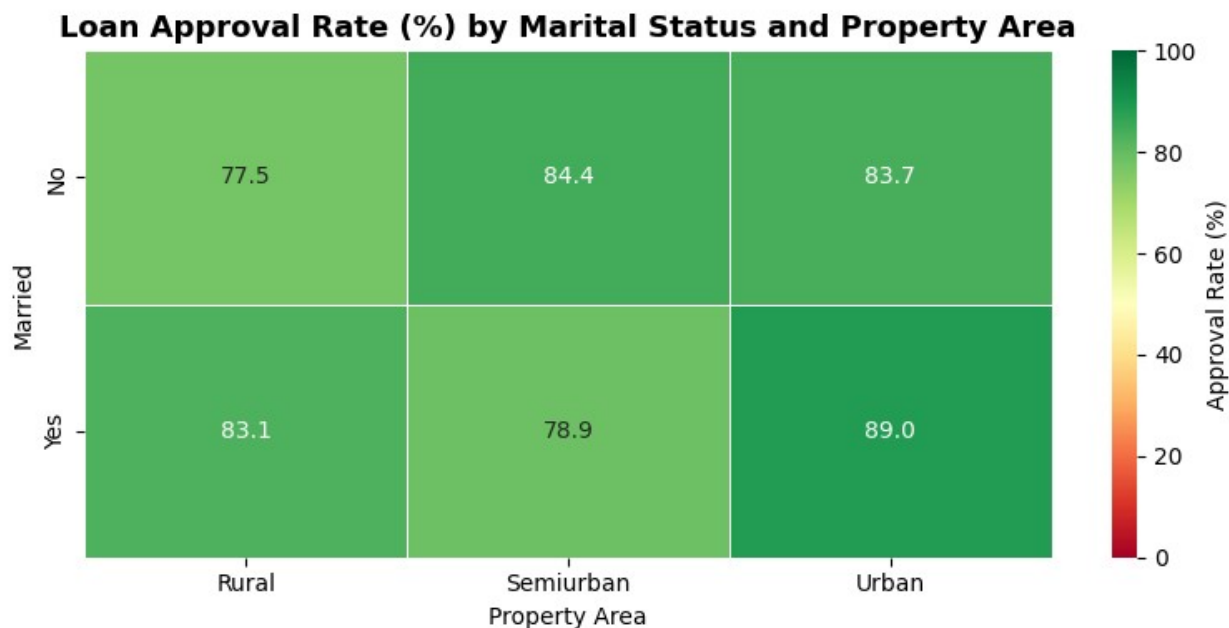


**Married People in Semiurban Area
(Total: 71)**



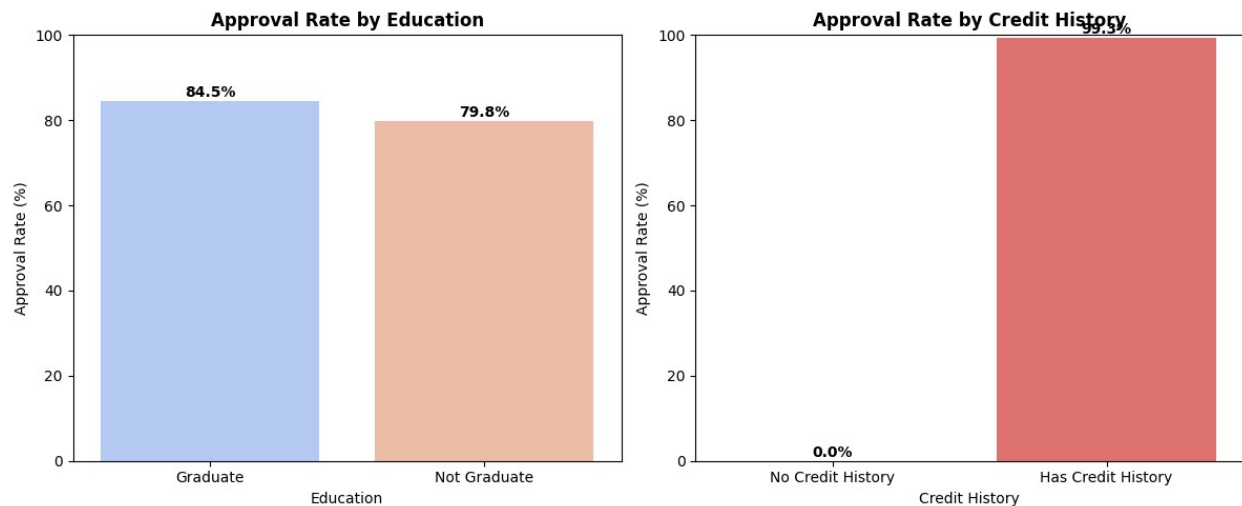
Plot 3 — Heatmap: Approval Rate by Marital Status × Property Area

A pivot heatmap crosses marital status (Married / Not Married) against property area (Rural, Semiurban, Urban) to display the predicted approval rate for every combination. This gives a clear view of which segment is most favoured by the model.



Plot 4 — Approval Rate by Education & Credit History

Two bar charts examine the effect of education level (Graduate / Not Graduate) and credit history (Has Credit History / No Credit History) on predicted loan approval. These plots confirm whether the feature importance patterns found during EDA hold up in predictions on new data.



13. Key Insights and Conclusion

13.1 Key Insights from New Customer Predictions

The following insights were drawn from analyzing the model's predictions on the new customer dataset:

- **Property Area:** Semiurban applicants showed the highest predicted approval rate across all three property area categories confirming the EDA pattern from the training data.
- **Marital Status:** Married applicants received a noticeably higher predicted approval rate compared to unmarried applicants.
- **Married in Semiurban Areas:** This subgroup had the highest approval rate of any combined segment, demonstrating that both marital status and property location are strong positive indicators.
- **Education:** Graduate applicants had a higher predicted approval rate than nongraduates matching the pattern seen in the training data EDA.
- **Credit History:** Applicants with a positive credit history had a significantly higher predicted approval rate. This is consistent with the correlation analysis finding that credit history is the strongest single predictor of loan approval.

13.2 Conclusion

This report documents project of the loan approval data mining project. The work covers the full end-to-end pipeline from raw data inspection through to prediction on new customer data.

The pipeline was carefully constructed to prevent data leakage: the train/test split is performed first, and all fitting operations categorical encoding, outlier bounds computation, median imputation, and feature scaling are fitted exclusively on the training set before being applied to the test set and new data.

Three classification models were trained, evaluated at baseline, and then tuned using GridSearchCV with 5-fold cross-validation optimizing the F1-score. Naive Bayes achieved the highest Test F1-Score after tuning and was selected as the final model for predicting loan status on new, unseen customers.

The predictions on new customer data validated the patterns uncovered in EDA. Credit history remains the most decisive single factor, while a combination of being married and residing in a semiurban area further increases the likelihood of loan approval according to the model.